Experiment no: 5

Experiment Name: Joins and Set Operations
Aim: Performing practical implementation of Joins and Set Operations.

PART B
(PART B: TO BE COMPLETED BY STUDENTS)

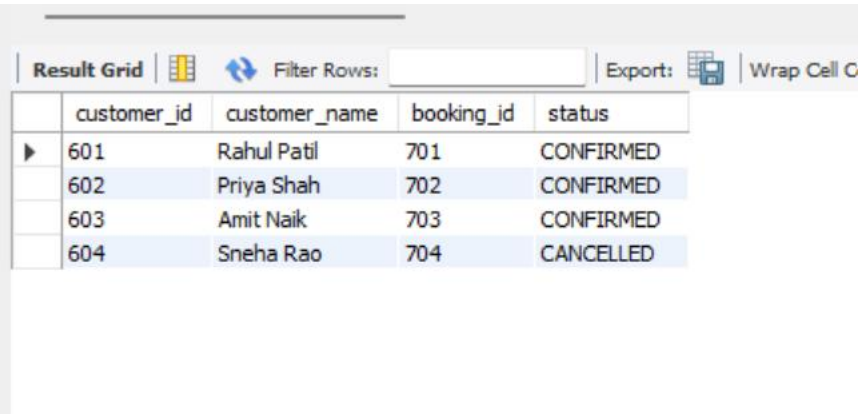
Roll. No: C-60	Name: Avdhut Madhavi
Class: C	Batch: C2
Date of Experiment:	Date of Submission:
Grade:	

B.1: Read the given question carefully and write a SQL statement and check the output. (Put the query and its output after each question.)

To perform practical implementation of SQL joins and set operations using the Cinema Ticket Booking System Database.

1. INNER JOIN — Display customers and their bookings.

```
SELECT c.customer_id,
    c.customer_name,
    b.booking_id,
    b.status
FROM customer c
INNER JOIN booking b
  ON c.customer_id = b.customer_id;
```

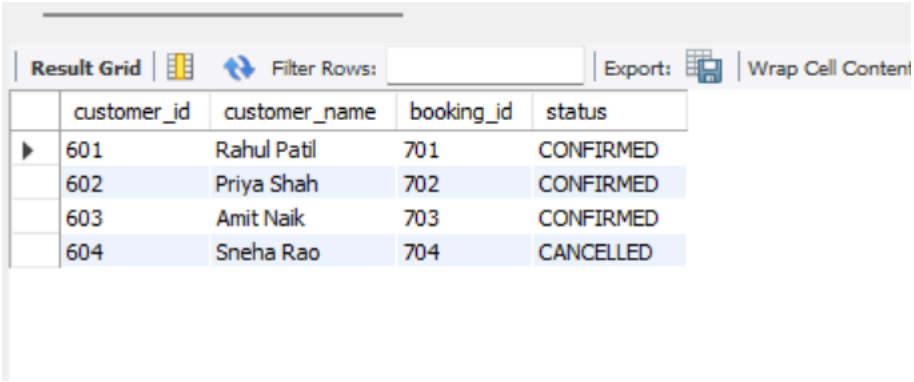


customer_id	customer_name	booking_id	status
601	Rahul Patil	701	CONFIRMED
602	Priya Shah	702	CONFIRMED
603	Amit Naik	703	CONFIRMED
604	Sneha Rao	704	CANCELLED

Explanation: Displays customers who have matching records in the booking table.

2. LEFT OUTER JOIN — Display all customers and their bookings

```
SELECT c.customer_id,
    c.customer_name,
    b.booking_id,
    b.status
FROM customer c
LEFT JOIN booking b
  ON c.customer_id = b.customer_id;
```

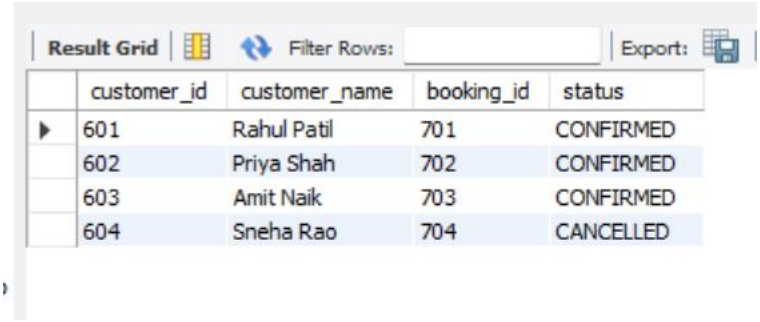


customer_id	customer_name	booking_id	status
601	Rahul Patil	701	CONFIRMED
602	Priya Shah	702	CONFIRMED
603	Amit Naik	703	CONFIRMED
604	Sneha Rao	704	CANCELLED

Explanation: Displays every customer, including customers who have not made any bookings. Booking columns show NULL when no matching booking exists.

3. RIGHT OUTER JOIN — Display all bookings and their customers

```
SELECT c.customer_id,
    c.customer_name,
    b.booking_id,
    b.status
FROM customer c
RIGHT JOIN booking b
  ON c.customer_id = b.customer_id;
```



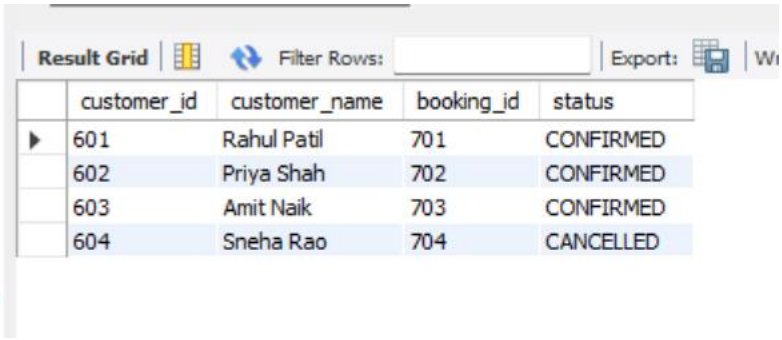
Result Grid | Filter Rows: | Export:

customer_id	customer_name	booking_id	status
601	Rahul Patil	701	CONFIRMED
602	Priya Shah	702	CONFIRMED
603	Amit Naik	703	CONFIRMED
604	Sneha Rao	704	CANCELLED

Explanation: Displays every booking and its matching customer, including any booking without a matching customer.

4. FULL OUTER JOIN — Display all customers and bookings
Note: - MySQL Workbench does not support FULL OUTER JOIN directly. We can simulate it using LEFT JOIN, RIGHT JOIN, and UNION.

```
SELECT c.customer_id,
    c.customer_name,
    b.booking_id,
    b.status
FROM customer c
LEFT JOIN booking b
  ON c.customer_id = b.customer_id
UNION
SELECT c.customer_id,
    c.customer_name,
    b.booking_id,
    b.status
FROM customer c
RIGHT JOIN booking b
  ON c.customer_id = b.customer_id;
```



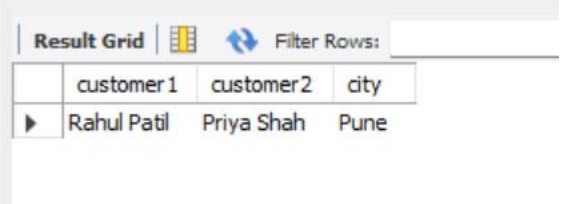
Result Grid | Filter Rows: | Export:

customer_id	customer_name	booking_id	status
601	Rahul Patil	701	CONFIRMED
602	Priya Shah	702	CONFIRMED
603	Amit Naik	703	CONFIRMED
604	Sneha Rao	704	CANCELLED

Explanation: Combines matched records with unmatched customers and unmatched bookings.

5. SELF JOIN — Find pairs of customers from the same city

```
SELECT c1.customer_name AS customer1,
       c2.customer_name AS customer2,
       c1.city
FROM customer c1
JOIN customer c2
   ON c1.city = c2.city
  AND c1.customer_id < c2.customer_id;
```



Result Grid | Filter Rows:

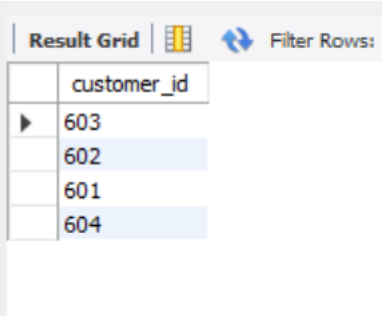
customer1	customer2	city
Rahul Patil	Priya Shah	Pune

Explanation: Joins the customer table with itself to find pairs of different customers who live in the same city.

6. UNION — Combine customer IDs and booked customer IDs

```
SELECT customer_id
FROM customer
UNION
SELECT customer_id
```

FROM booking;



Explanation: Combines customer IDs from both tables and removes duplicate IDs.

7. UNION ALL — Combine IDs while retaining duplicates

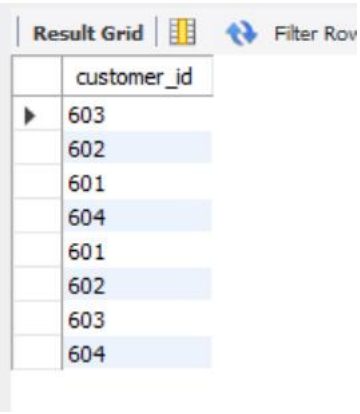
SELECT customer_id

FROM customer

UNION ALL

SELECT customer_id

FROM booking;



Explanation: Combines both lists while retaining duplicates. Customers who have bookings can appear more than once.

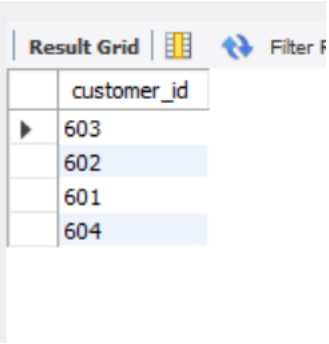
8. INTERSECT — Find customers who have made bookings
Note: - To make this query compatible with MySQL versions that do not support INTERSECT, use an inner join

SELECT DISTINCT c.customer_id

FROM customer c

INNER JOIN booking b

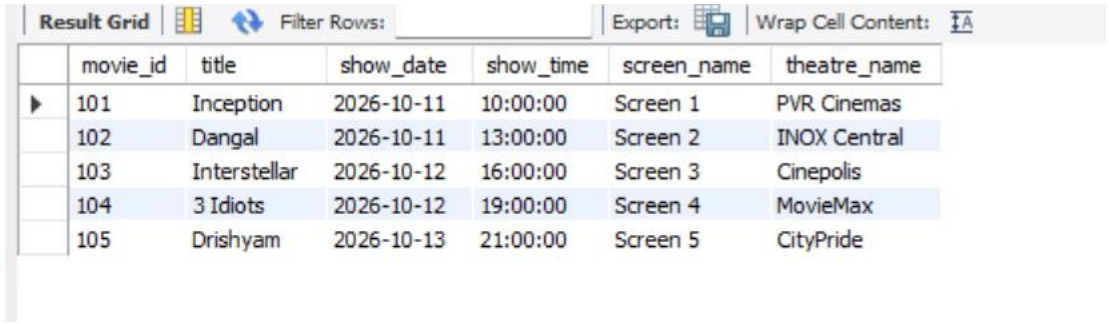
ON c.customer_id = b.customer_id;



Explanation: Displays customer IDs that occur in both tables, identifying customers who have bookings.

9. Join movie, showtime, screen, and theatre tables

```
SELECT m.movie_id,
    m.title,
    s.show_date,
    s.show_time,
    sc.screen_name,
    t.theatre_name
FROM movie_details m
JOIN showtime s
  ON m.movie_id = s.movie_id
JOIN screen sc
  ON s.screen_id = sc.screen_id
JOIN theatre t
  ON sc.theatre_id = t.theatre_id;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

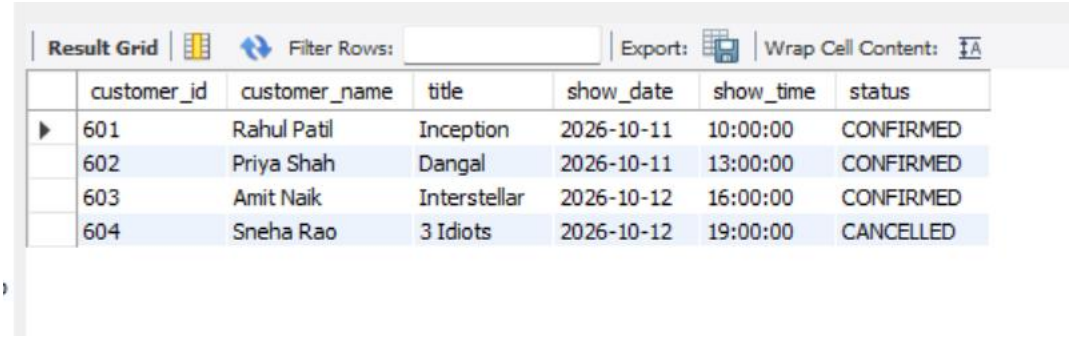
movie_id	title	show_date	show_time	screen_name	theatre_name
101	Inception	2026-10-11	10:00:00	Screen 1	PVR Cinemas
102	Dangal	2026-10-11	13:00:00	Screen 2	INOX Central
103	Interstellar	2026-10-12	16:00:00	Screen 3	Cinepolis
104	3 Idiots	2026-10-12	19:00:00	Screen 4	MovieMax
105	Drishyam	2026-10-13	21:00:00	Screen 5	CityPride

Explanation:

- movie_details provides movie information.
- showtime provides the show date, time, and ticket price.
- screen provides the screen name.
- theatre provides the theatre name.

10. Join customers with their booked movies

```
SELECT c.customer_id,
    c.customer_name,
    m.title,
    s.show_date,
    s.show_time,
    b.status
FROM customer c
JOIN booking b
  ON c.customer_id = b.customer_id
JOIN showtime s
  ON b.show_id = s.show_id
JOIN movie_details m
  ON s.movie_id = m.movie_id;
```



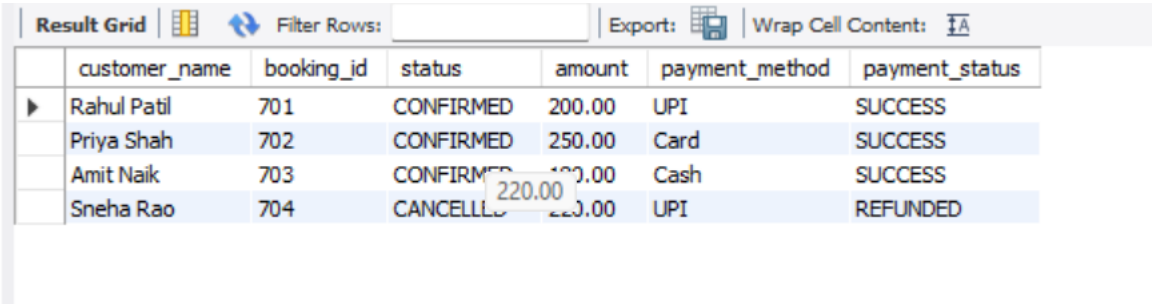
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	title	show_date	show_time	status
601	Rahul Patil	Inception	2026-10-11	10:00:00	CONFIRMED
602	Priya Shah	Dangal	2026-10-11	13:00:00	CONFIRMED
603	Amit Naik	Interstellar	2026-10-12	16:00:00	CONFIRMED
604	Sneha Rao	3 Idiots	2026-10-12	19:00:00	CANCELLED

Explanation: Displays each customer's booked movie, show date, show time, and booking status.

11. Join bookings with their payment details

```
SELECT c.customer_name,
       b.booking_id,
       b.status,
       p.amount,
       p.payment_method,
       p.payment_status
FROM customer c
JOIN booking b
   ON c.customer_id = b.customer_id
JOIN payment p
   ON b.booking_id = p.booking_id;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	booking_id	status	amount	payment_method	payment_status
Rahul Patil	701	CONFIRMED	200.00	UPI	SUCCESS
Priya Shah	702	CONFIRMED	250.00	Card	SUCCESS
Amit Naik	703	CONFIRMED	220.00	Cash	SUCCESS
Sneha Rao	704	CANCELLED	220.00	UPI	REFUNDED

Explanation: Displays customer details, booking information, and the corresponding payment details.

B.2: Write an observation according to the query execution

1. SQL joins were used to retrieve related information from multiple tables in the Cinema Ticket Booking System.
2. INNER JOIN returned matching records from both tables.
3. LEFT JOIN and RIGHT JOIN demonstrated how unmatched records are handled.
4. FULL OUTER JOIN functionality was simulated using LEFT JOIN, RIGHT JOIN, and UNION because MySQL does not directly support FULL OUTER JOIN.
5. SELF JOIN was used to compare customers living in the same city.
6. UNION combined query results while removing duplicates.
7. UNION ALL combined query results while retaining duplicates.

8. Multiple-table joins were used to display movie schedules, theatre details, customer bookings, and payment information.

Result: SQL joins and set operations were practised using MySQL Workbench.

B.3: Write an answer of the following questions

1.What is a JOIN? Explain types of joins.

A JOIN combines rows from two or more tables using a related column or join condition.

- INNER JOIN: Returns records that match in both tables.
- LEFT OUTER JOIN: Returns all records from the left table and matching records from the right table.
- RIGHT OUTER JOIN: Returns all records from the right table and matching records from the left table.
- FULL OUTER JOIN: Returns matched and unmatched records from both tables.
- SELF JOIN: Joins a table with itself to compare records within that table.

2.Difference between INNER JOIN and OUTER JOIN.

INNER JOIN	OUTER JOIN
Returns matching records only.	Includes matching and selected unmatched records.
Excludes rows without a match.	Preserves rows from one or both tables, depending on the join type.
Example: customers with matching bookings.	Example: all customers, including those without bookings.

3.What is SELF JOIN and when to use it?

A SELF JOIN joins a table with itself using different aliases. It is used to compare records within the same table.

Example: Finding customers from the same city.

```
SELECT c1.customer_name,
    c2.customer_name,
    c1.city
FROM customer c1
JOIN customer c2
  ON c1.city = c2.city
 AND c1.customer_id < c2.customer_id;
```

4.Difference between UNION and UNION ALL.

- UNION combines two query results and removes duplicate rows.
- UNION ALL combines two query results and retains duplicates.

Both queries must return the same number of columns with compatible data types

Q5. What is INTERSECT? What is MINUS?

- **INTERSECT:** Returns rows common to both query results.
- **MINUS:** Returns rows present in the first query result but absent from the second. In MySQL, EXCEPT is used for this operation instead of MINUS.

Example 1: Find customers who have made bookings.

```
SELECT customer_id
FROM customer
INTERSECT
SELECT customer_id
FROM booking;
```

Example 2: Find customers who have not made bookings.

```
SELECT customer_id
FROM customer
EXCEPT
SELECT customer_id
FROM booking;
```